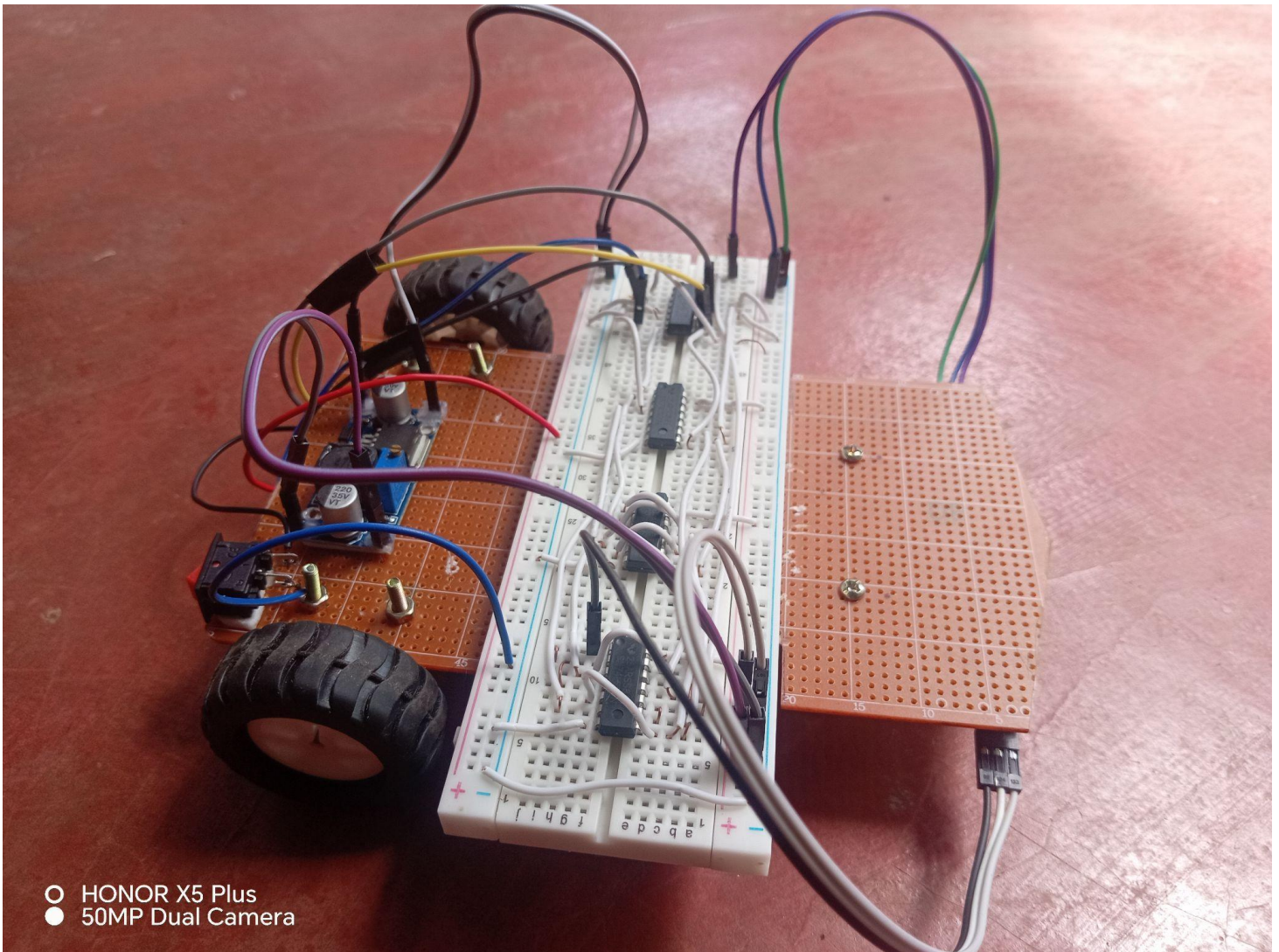


Team 20 - Line Following Robot Portofolio



Team 20 — Team Members

Kosala Bandara - E/23/027

Pasindu Bhanuka - E/23/028

Pamod Bandara - E/23/031

Sachin Bandara - E/23/037

Osanda Bandaranayake - E/23/038

Senuda Batawala - E/23/039

A line-following robot is a basic autonomous system designed to move along a predefined path by detecting and tracking a visible line on the ground. Although most contemporary designs depend on microcontrollers and software-based control algorithms, this project takes a different approach by focusing entirely on hardware implementation.

In this work, a line-following robot is designed and developed using only digital logic gates, without the use of any microcontroller or programmable device. The system interprets inputs from sensors through combinational logic circuits to produce real-time motor control signals. By directly linking sensor outputs to motor actions using logic gates, the robot operates through a purely hardware-driven control mechanism.

The main objective of this project is to investigate fundamental digital design concepts and demonstrate how autonomous behavior can be achieved using basic electronic components. This approach not only strengthens understanding of digital logic principles but also highlights an alternative method for developing intelligent systems without relying on software.

There are three steps:

1. Identify the path

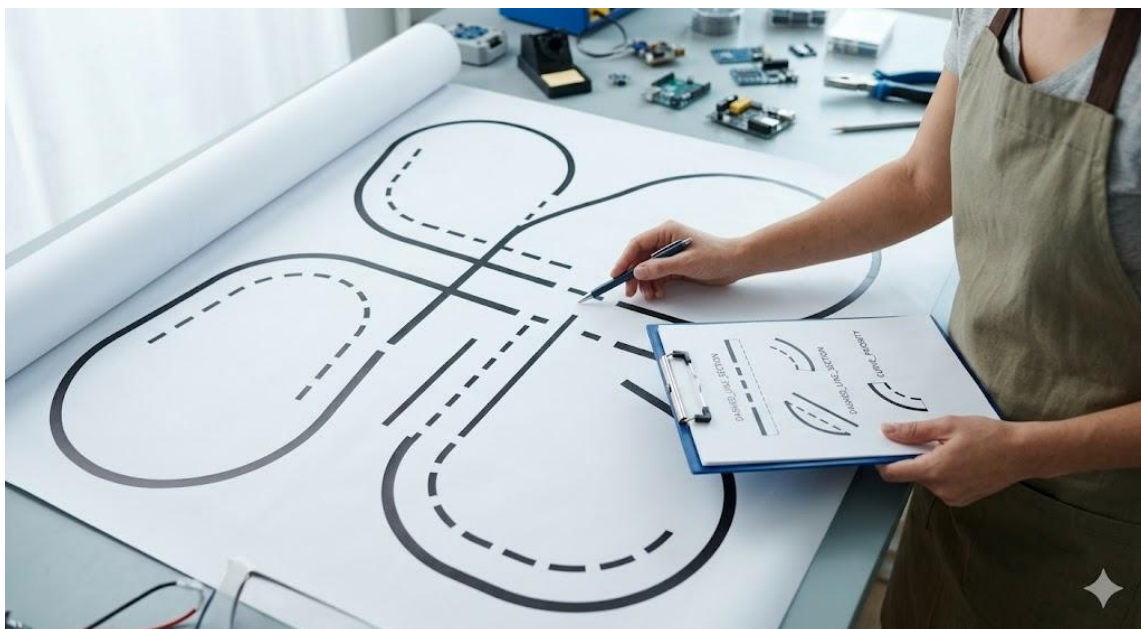


Figure 1

2. Design circuit



Figure 2

3. Simulate using proteus

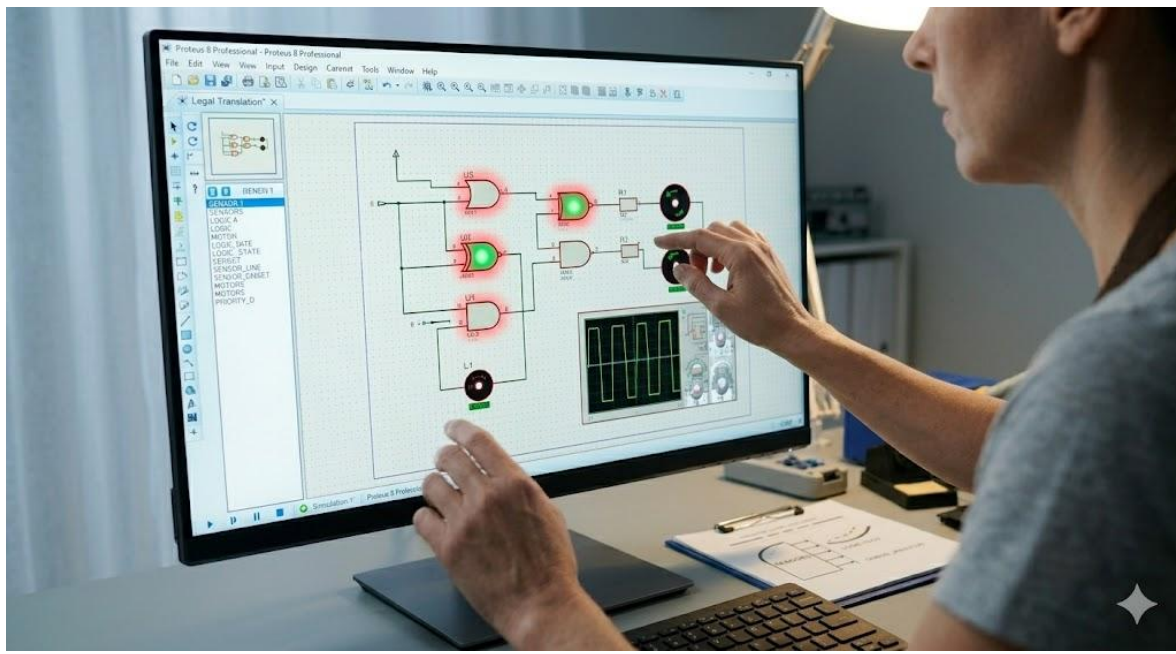


Figure 3

4. Make robot

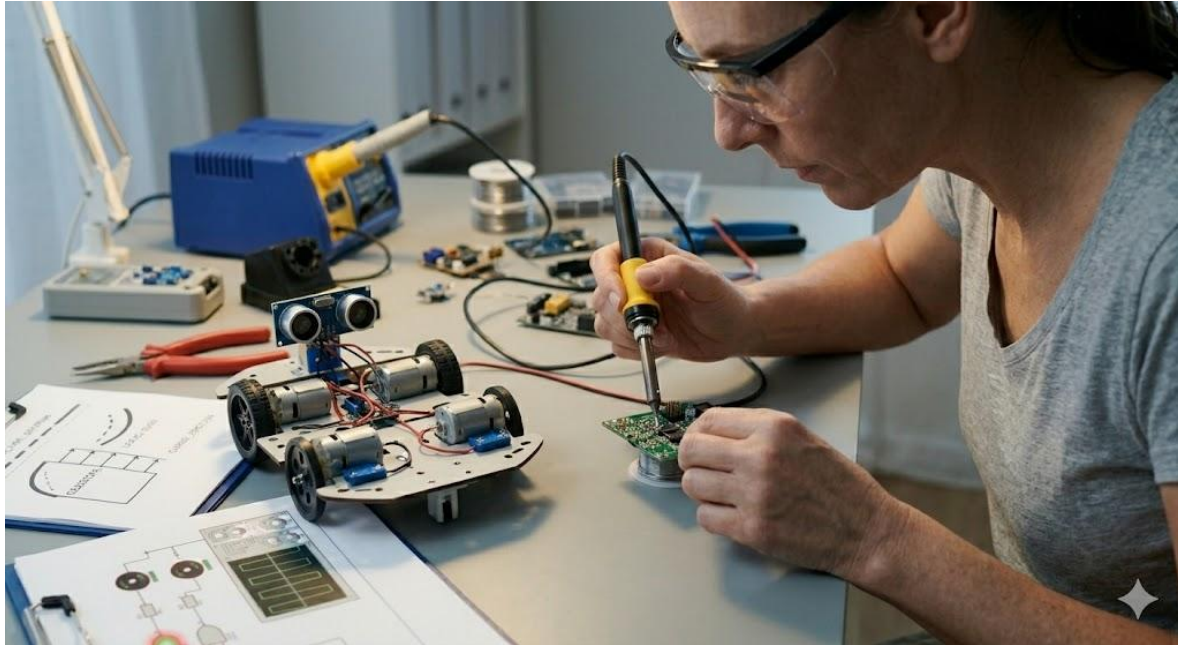


Figure 4

5. Drive



Figure 5

Day 01



Activity

- Visual analysis and strategic mapping of the physical line-following competition course to identify distinct navigational hazards, sensor edge cases, and mechanical challenges.

Objectives

- To break down the continuous track into distinct "challenge zones" that require specific logic or hardware solutions.
- To identify areas where standard real-time combinational logic might fail (e.g., broken lines, obscured paths).

- To evaluate the physical terrain to ensure the robot's mechanical design (clearance, traction) is adequate.
- To correlate previously designed logic features (like memory states and timers) to specific track obstacles.

Description of Track Challenges

1) Discontinuous/Dashed Lines ("Zone 4 - Land of Crocodiles")

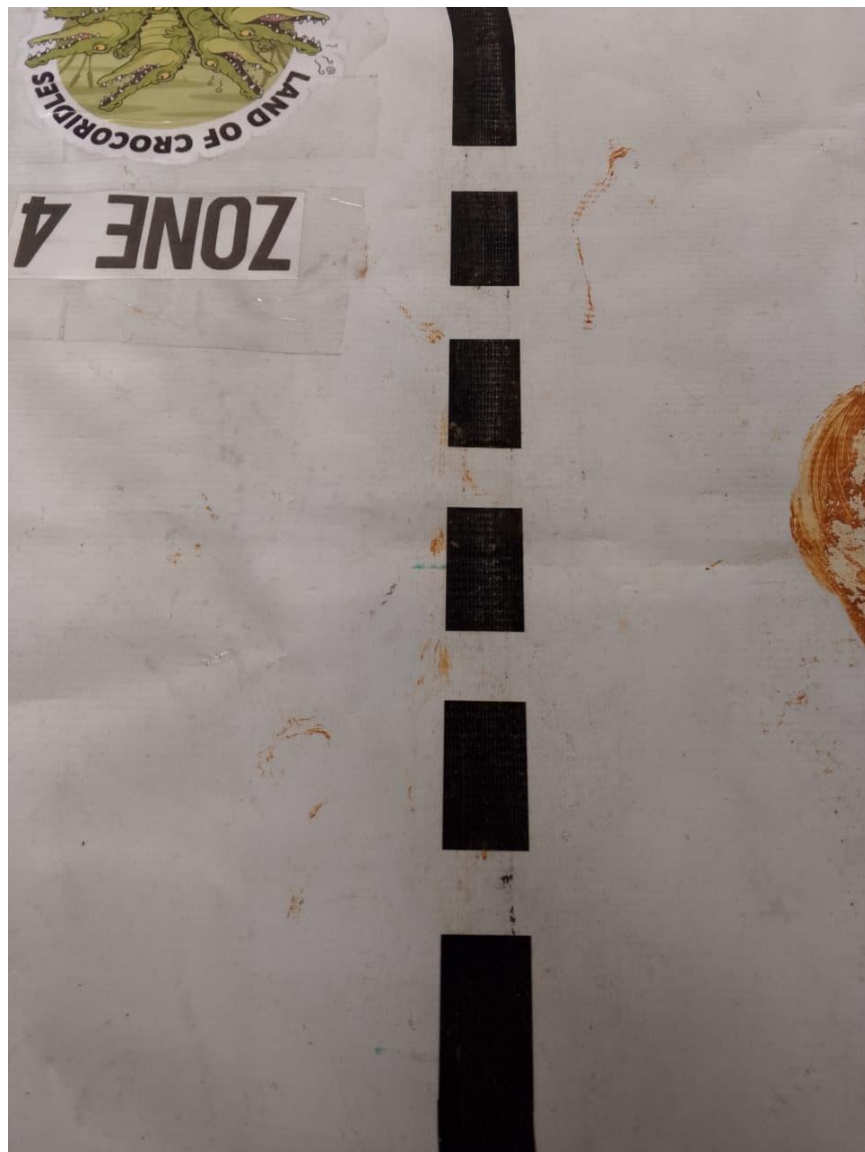


Figure 1:

- Located in "Zone 4 - Land of Crocodiles."

- Solid line breaks into small dashed segments.
 - Triggers "track lost" (0-0) sensor readings in the gaps.
 - Requires the previously designed **memory logic (D-latch)** to maintain forward momentum and prevent erratic spinning.
- 2) High-Frequency Serpentine Curves ("Zone 3 - Bends of Devil")



Figure 2:

- Located in "Zone 3 - Bends of Devil."
- Features continuous, tight S-curves with no straight recovery paths.
- Requires rapid, consecutive directional changes (left-right-left).
- Tests the responsiveness of the logic gates and the immediate reversal capabilities of the H-bridge motor driver.

3) Sensor Obscuration (Low-Contrast Zones)



Figure 3:

- A section of the track is covered in a brown, painted "mud" texture.
- Reduces the IR reflectivity contrast compared to clean black-on-white.
- May cause false readings or sensor "blindness."
- Requires careful physical calibration of the IR sensor threshold dials to function reliably in low-contrast conditions.
- changes as the robot tilts over the bump.

4) Sharp Geometries and Momentum Management



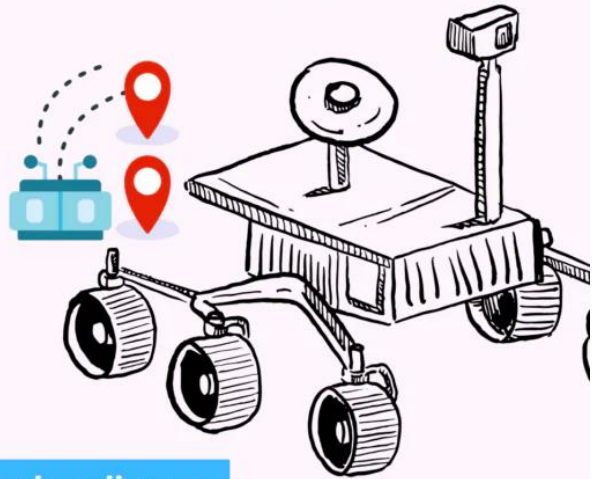
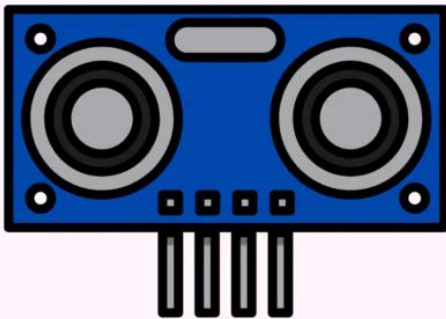
Figure 4:

- The overall layout mixes long straights with sharp 90-degree turns.
- Long straights allow the robot to reach maximum speed.
- Sharp corners test the robot's ability to overcome forward momentum and pivot instantly.
- Demands a careful balance of battery voltage (speed) and sensor placement width (reaction time).

Day 02



Line Follower Robot



Get Started With Techradiance

Status **Completed**

Date Dec 31, 2025 4:00 PM

Activity

- Concept introduction and Sensor Logic Development

Objectives

- To understand the working principle of a line following robot and to evaluate different sensor configurations for effective path tracking

Description

1) What we discussed

- Initially considered a **two-sensor configuration** (left and right sensors).
- Discussed basic control logic:
- Evaluated system performance during testing and team discussions.
- Explored the need for improved accuracy and stability in line detection.

2) Decisions Made

- Decided to move from a two-sensor system to a three-sensor system.
- Added a **central sensor** to improve line tracking.
- Updated control logic to incorporate feedback from all three sensors.
- prioritized better handling of turns and alignment over system simplicity.

3) Problems Faced

- Difficulty in handling sharp turns with only two sensors.
- Inability to **detect complete loss of the line** reliably.
- Poor alignment accuracy (robot could drift off the line).
- Limited feedback from only two sensors made control decisions less precise.
- Increased chances of oscillation (zig-zag movement) during motion.

4) Outcome

- Improved **line detection accuracy** with the addition of the center sensor.
- Better **alignment control**, especially on straight paths.
- Enhanced performance during **turning maneuvers**.
- Reduced chances of losing the line completely.
- More stable and smoother movement of the robot.

5) Optional Enhancement can made

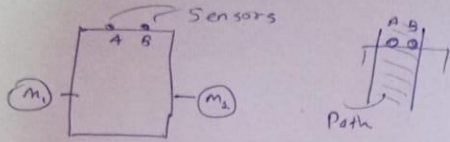
- System became more robust and reliable under varying track conditions.
- The design is now more scalable (can extend to 5 or 8 sensors later, like QTR arrays).
- Provides a foundation for implementing advanced control methods (e.g., PID control).
- Slight increase in complexity, but justified by significant performance improvement.

6) Conclusion

- On Day 01, the two-sensor and three-sensor approaches were evaluated. The three-sensor configuration was selected because it offers better accuracy and greater stability for line-following applications.

2025.12.31

① 2 Sensors



A	B	M ₁	M ₂
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

$$M_1 = \bar{A}B + AB = B(\bar{A} + A)$$

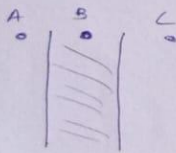
$$M_1 = \underline{\underline{B}}$$

$$M_2 = \bar{A}B + AB$$

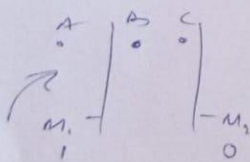
$$= A(\bar{B} + B)$$

$$M_2 = \underline{\underline{A}}$$

② 3 Sensors



A	B	C	M ₁	M ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	X	X
1	1	0	0	1
1	1	1	X	X



AB	00	01	11	10
C	0	0	1	0
1	1	1	X	X

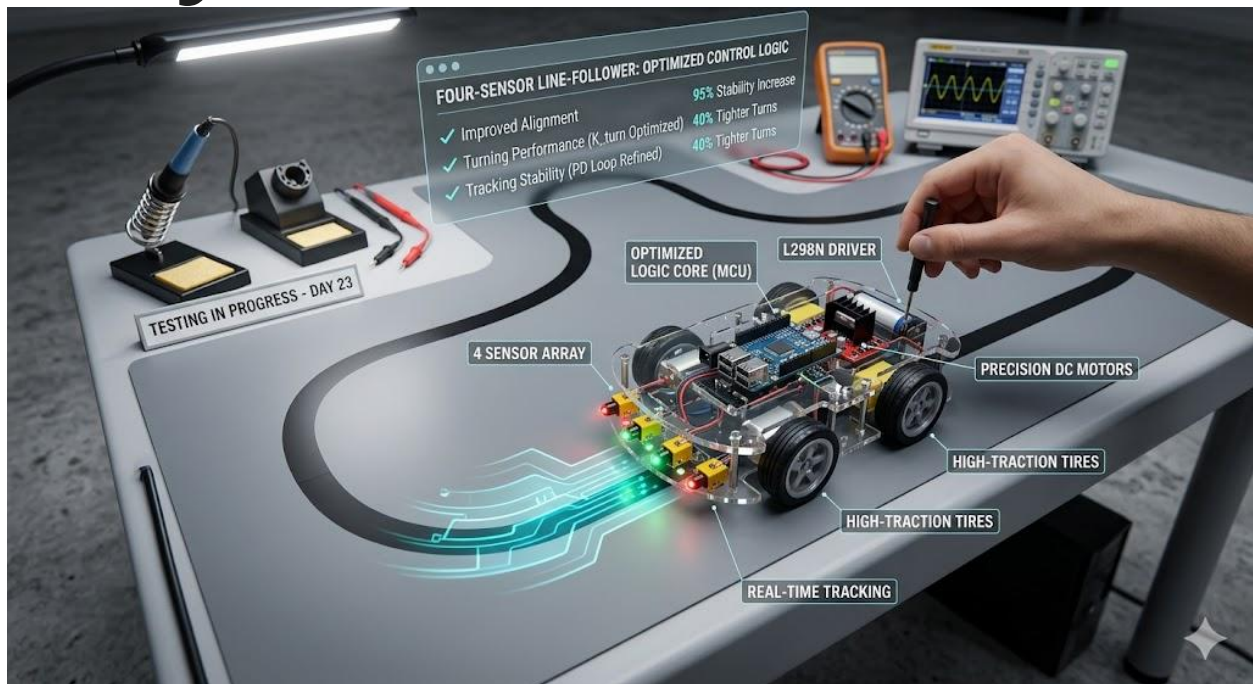
$$M_1 = \bar{A}B + C$$

AB	00	01	11	10
C	0	0	1	1
1	0	0	X	X

$$M_2 = A + B\bar{C}$$

Figure 5: Two & Three sensor truth tables and boolean expressions

Day 03



Activity

- Designed, implemented, and tested a four-sensor line-following system with optimized control logic to improve alignment, turning performance, and overall tracking stability.

Objectives

- To **improve line detection accuracy** by increasing the number of sensors.
- To achieve **smoother and more stable movement** of the robot.
- To enhance the robot's ability to **handle sharp turns and curved paths** effectively.

- To **reduce the probability of losing the line** during operation.
- To develop a more **responsive and robust control system** using multiple sensor inputs.
- To ensure reliable performance under **varying track conditions**.

Description

1) Concept Discussed

- Considered a **four-sensor configuration**:
 - Two sensors on the left side.
 - Two sensors on the right side.
- Designed control logic based on sensor roles:
 - **Middle sensors** → maintain forward motion.
 - **Outer sensors** → detect slight deviations and trigger small corrections.
 - **Inner sensors losing alignment** → trigger sharper turns.
- Discussed how increasing sensor count could improve positional awareness.

2) Design Decisions

- Adopted the four-sensor configuration to improve tracking accuracy.
- Developed enhanced control logic using multiple sensor inputs.
- Assigned specific roles to sensors (center for alignment, outer for correction).
- Considered path variations (curves, sharp turns) while finalizing the design.

- Decided to prioritize smooth navigation and stability over hardware simplicity.

3) Problems Identified (from earlier configurations & testing)

- Limited accuracy in detecting exact robot position relative to the line.
- Difficulty handling sharp curves smoothly.
- Higher risk of completely losing the line with fewer sensors.
- Abrupt or unstable movements due to insufficient feedback.

4) Improvements Observed

- **Finer resolution** in detecting position relative to the line.
- Smoother and more **gradual corrections** during movement.
- Improved handling of **sharp turns and curves**.
- Continuous monitoring by multiple sensors reduces error.
- Better **responsiveness** to path deviations.

5) Final Outcome

- Significant improvement in accuracy, stability, and control.
- Robot performs **sharp turns smoothly**.
- Reduced probability of **losing the line completely**.
- Overall system is more **robust and reliable** under varying path conditions.

6) Additional Enhancements

- Configuration supports future upgrade to sensor arrays (e.g., 5 or 8 sensors).
- Enables implementation of advanced control algorithms (like PID).
- Improves adaptability to complex tracks and intersections.
- Slight increase in hardware complexity, but performance gain is substantial.

Line following Final sensor configuration



A	B	C	D	Left		Right	
				In1	In2	In3	In4
0	0	0	0	1	0	1	0
0	0	0	1	1	0	0	1
0	0	1	0	1	0	0	0
0	0	1	1	1	0	0	0
0	1	0	0	0	0	1	0
0	1	0	1	x	x	x	x
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	0
1	0	0	0	0	1	1	0
1	0	0	1	x	x	x	x
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	0	0	1	0
1	1	0	1	x	x	x	x
1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0

I_{n1}

$\begin{array}{c} AB \\ \swarrow \searrow \\ C D \end{array}$	00	01	11	10
00	1	0	0	0
01	1	X	X	X
11	1	1	0	X
10	1	1	1	X

$$I_{n1} = \bar{A}\bar{B} + C\bar{D} + \bar{A}D$$

I_{n2}

$\begin{array}{c} AB \\ \swarrow \searrow \\ C D \end{array}$	00	01	11	10
00	0	0	0	1
01	0	X	X	X
11	0	0	0	X
10	0	0	0	X

$$I_{n2} = A\bar{B}$$

I_{n3}

$\begin{array}{c} AB \\ \swarrow \searrow \\ C D \end{array}$	00	01	11	10
00	1	1	1	1
01	0	X	X	X
11	0	1	X	1
10	0	0	X	1

~~$$I_{n3} = A + \bar{C}\bar{D} + BD$$~~

J_{n3}

$AB \backslash CD$	00	01	11	10
00	1	1	1	1
01	0	X	X	X
11	0	1	1	X
10	0	0	1	X

$$J_{n3} = A + \bar{C}\bar{D} + BD$$

J_{n4}

$AB \backslash CD$	00	01	11	10
00	0	0	0	0
01	1	X	X	X
11	0	0	0	X
10	0	0	0	X

$$J_{n4} = \bar{C}D$$

J_{n3}

$\begin{matrix} AB \\ \backslash CD \end{matrix}$	00	01	11	10
00	1	1	1	1
01	0	X	X	X
11	0	1	1	X
10	0	0	1	X

$$J_{n3} = A + \bar{C}\bar{D} + BD$$

J_{n4}

$\begin{matrix} AB \\ \backslash CD \end{matrix}$	00	01	11	10
00	0	0	0	0
01	1	X	X	X
11	0	0	0	X
10	0	0	0	X

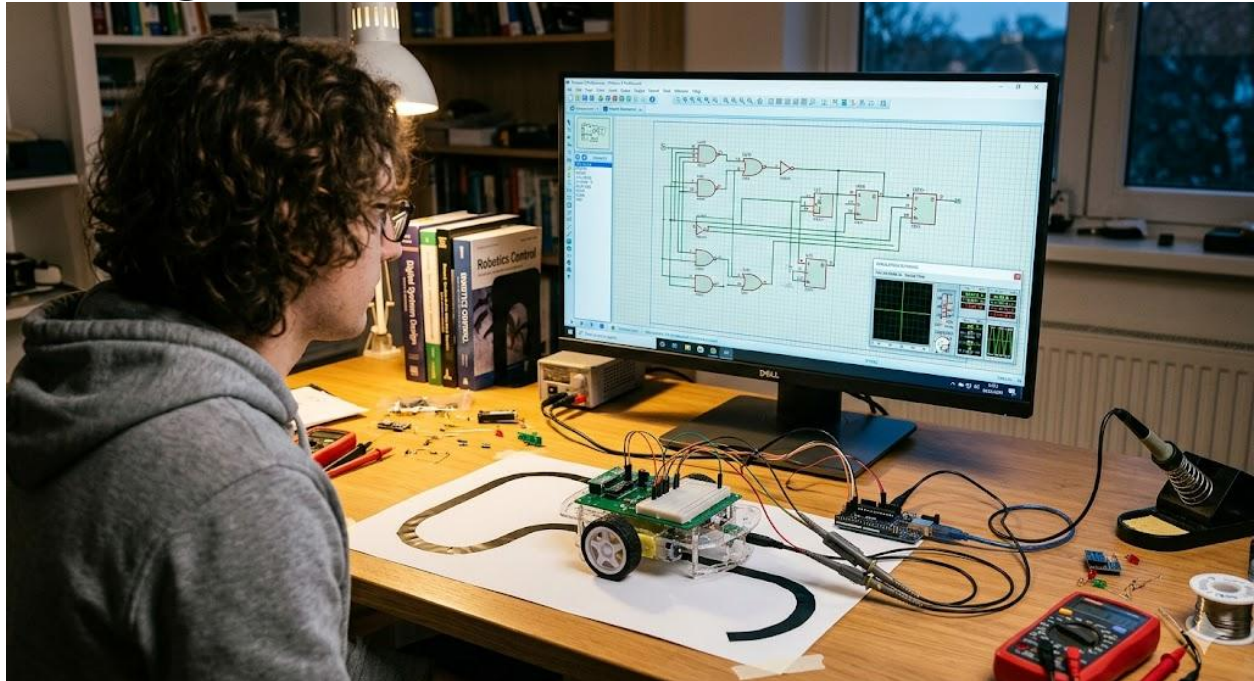
$$J_{n4} = \bar{C}D$$

Figure 6

Conclusion

- In day 3, as a conclusion we thought to implement all the previous sensor configurations in Proteus Software and observe the output variation. The given path has dashed lines also, hence in team discussions we decided to apply flip flop for memory configuration.

Day 04



Activity

- Designing, analyzing, and simulating logic-gate-based control systems for a line-following robot using **Proteus 8 Professional**.

Objectives

- To design and test combinational logic circuits for direct, real-time sensor-to-motor control.
- To implement sequential logic (memory) using components like D-latches to improve robot navigation during edge cases (e.g., losing the track).
- To utilize J-K flip-flops and clock signals to create state-based motor control systems.

- To successfully interface logic gate outputs with hardware driver simulations, specifically the L298 motor driver, to control continuous DC motor movement and direction.

Description

1) Combinational Control Configurations (Without Memory)

- Utilizes 3-sensor and 4-sensor arrays.
- Relies entirely on basic combinational logic gates (AND, OR, NOT).
- Reactions are instantaneous based purely on real-time sensor readings.
- Lacks a memory component; if all sensors lose the line (e.g., a 0-0 reading), the robot cannot remember its previous position and may spin aimlessly.

2) Memory Integration using D-Latches (2-Sensor Configuration)

- Introduces a **74LS75 D-latch** to the basic logic gate circuit.
- The D-latch functions as a built-in memory module.
- During standard operation, real-time inputs continue to steer the robot.
- If the track is lost (both sensors read 0-0), the latch instantly freezes to "remember" which side last detected the line.
- Logic gates use this stored state to pivot the robot back toward the track.

3) Advanced State-Based Control using J-K Flip-Flops (4-Sensor Configuration)

- Upgrades the system to sequential logic using **J-K flip-flops**.

- The four sensor inputs (A, B, C, D) do not directly force motor states.
- Sensor inputs pass through logic gates to configure the **J** and **K** input pins.
- The system operates on a clock pulse; flip-flops read the J and K pins at each trigger to determine the *next* action for the motors.

4) Interfacing with the L298 Motor Driver

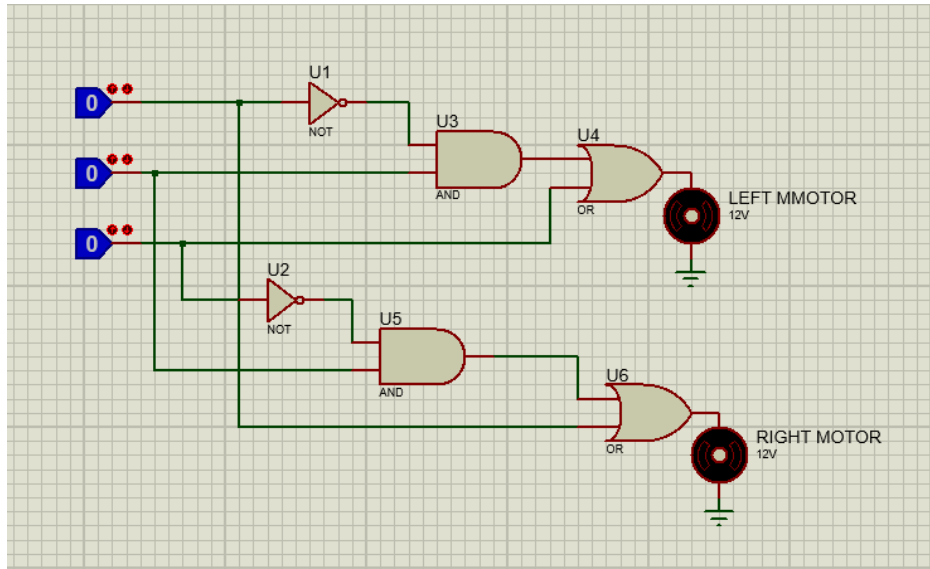
- Connects the logic circuit outputs to an **L298 motor driver** to simulate hardware wheel control.
- The driver's Enable pins (ENA/ENB) are permanently tied to High (VCC).
- Motor directional pins are controlled by the mutually exclusive Q and Qbar outputs of the flip-flops.
- Because Q and Qbar are always opposites (1 and 0), the motors never stop spinning; they only continuously swap directions based on the flip-flop state.

5) Logic Simplification via Karnaugh Maps

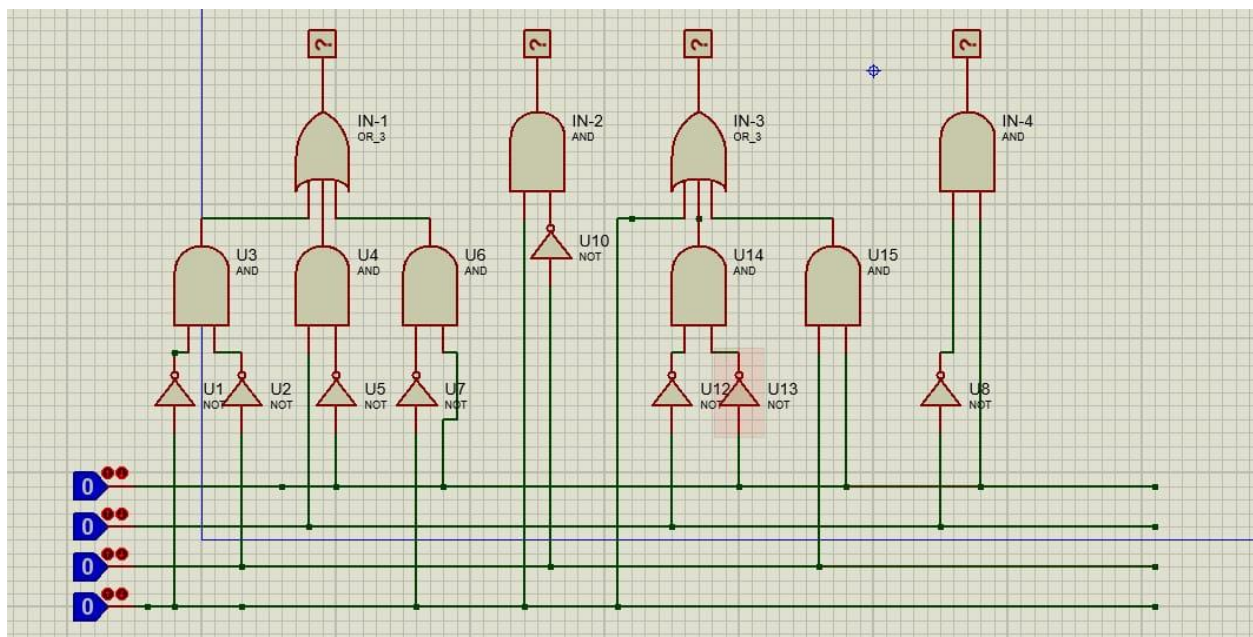
- Involves optimizing the boolean logic before building the circuits in Proteus.
- Utilizes **Karnaugh maps (K-maps)** to find the simplest logic expressions for the motor driver inputs.
- Results in derived equations (ex, $In_1 = \{A\}\bar{Z}_k + B$) that dictate the gate layout.

Simulations(Proteus 8 Professional)

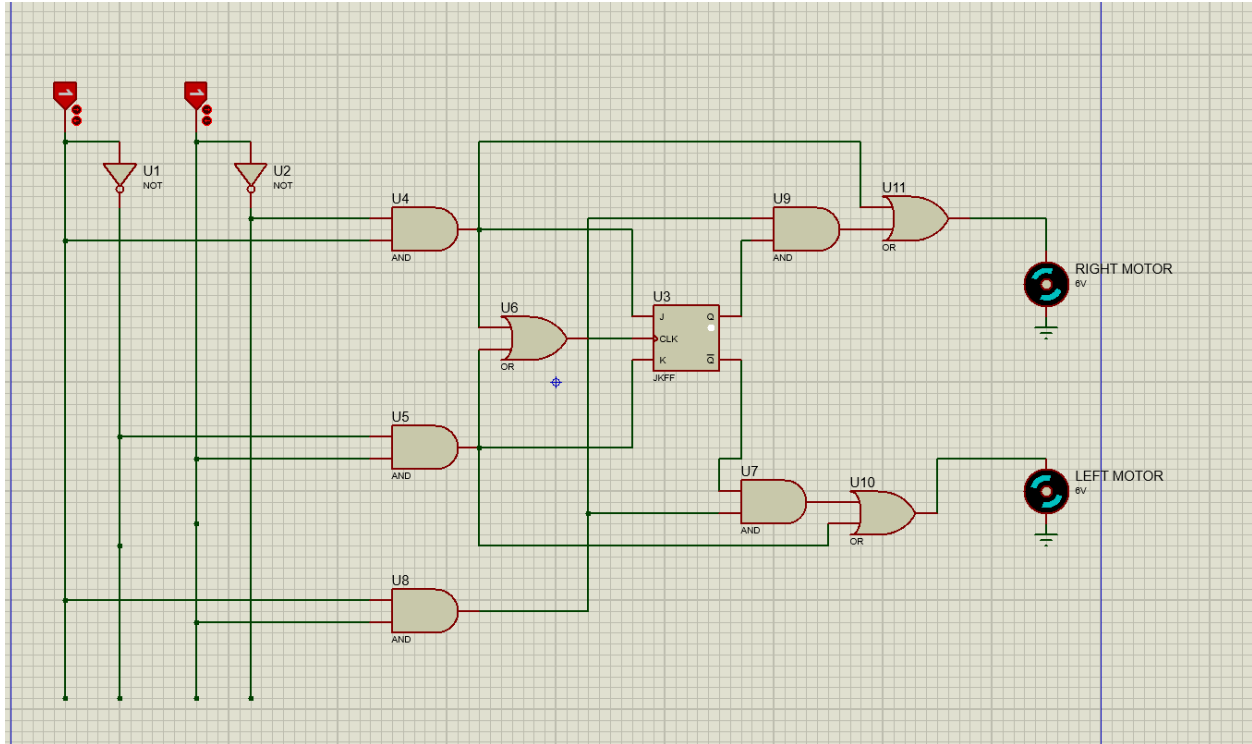
➤ 3 Sensor Configuration(Without Memory)



➤ 4 Sensor Configuration(Without Memory)



➤ 2 Sensor Configuration(With 1 J-K Flip Flop)



Simulation Video: [Click here](#)

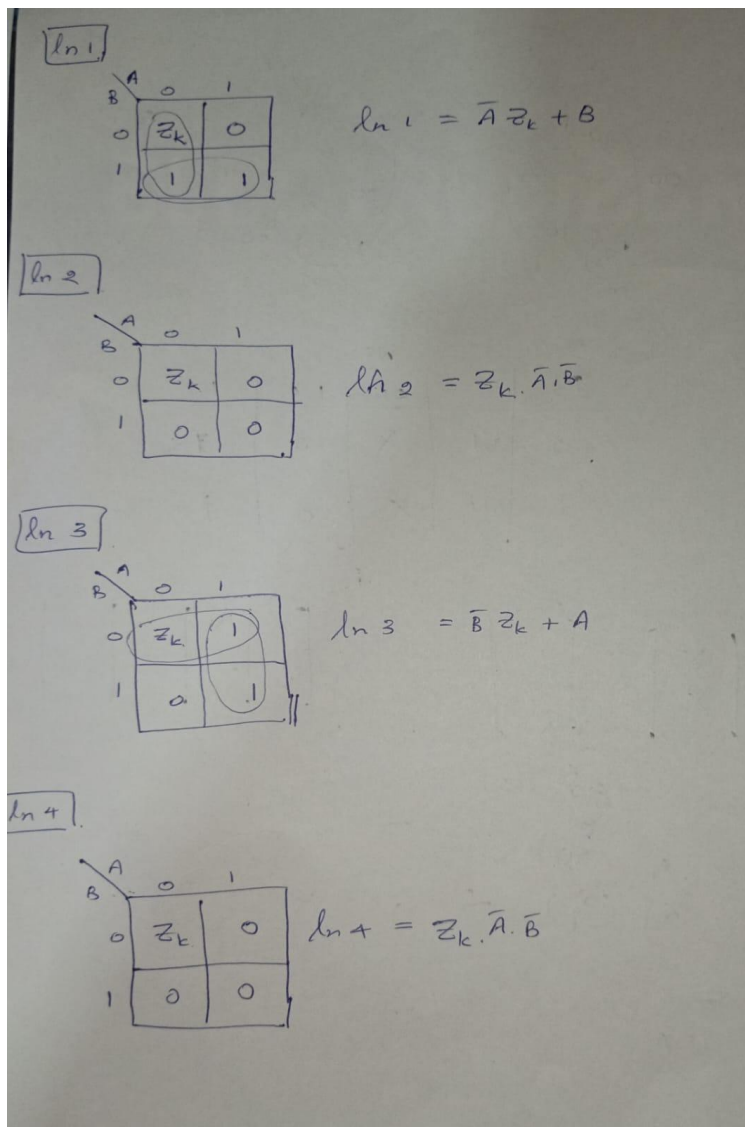
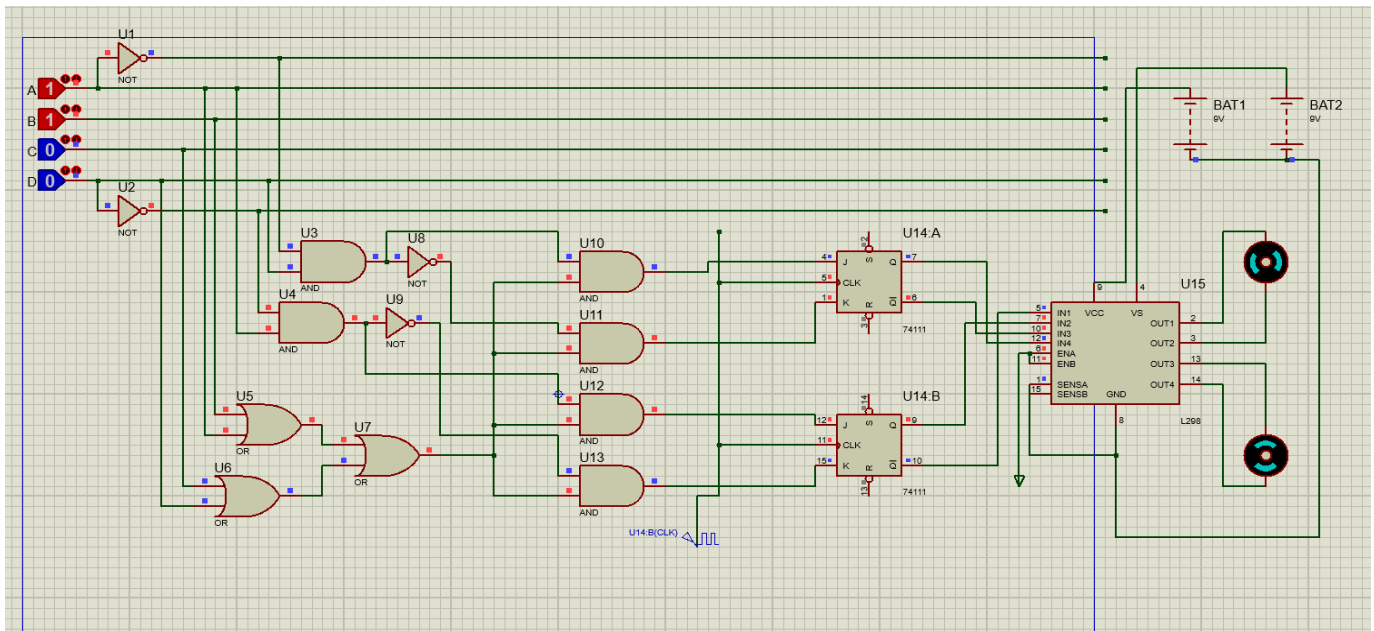
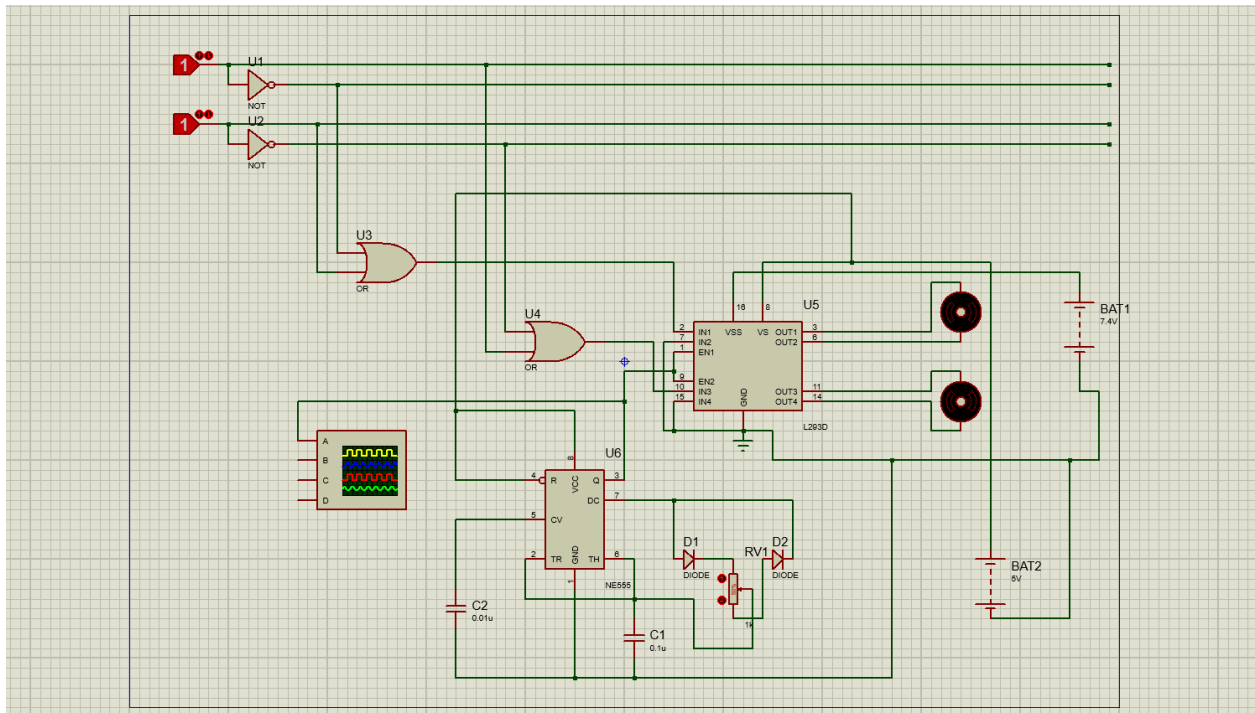


Figure 6 - Usage of previous memory to 2 sensor configuration

➤ 4 Sensor Configuration(With 2 J-K Flip Flops)

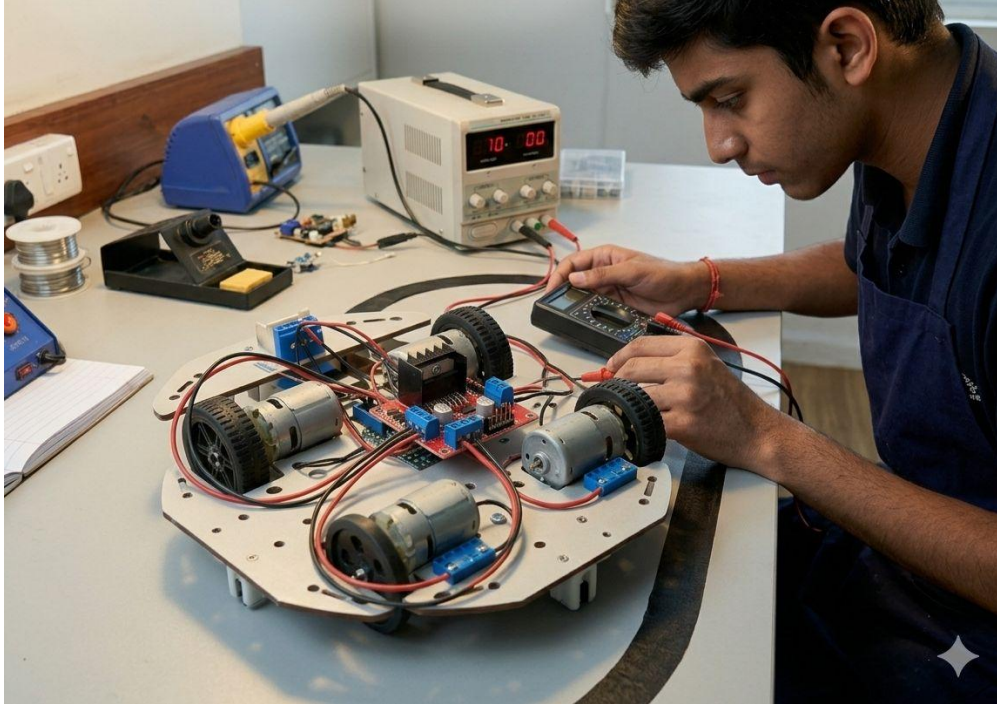


➤ 2 Sensor Configuration(With PWM Pulse from NE555 'TIMER' for change the speed)



Simulation Video: [Click here](#)

Day 05



Activity

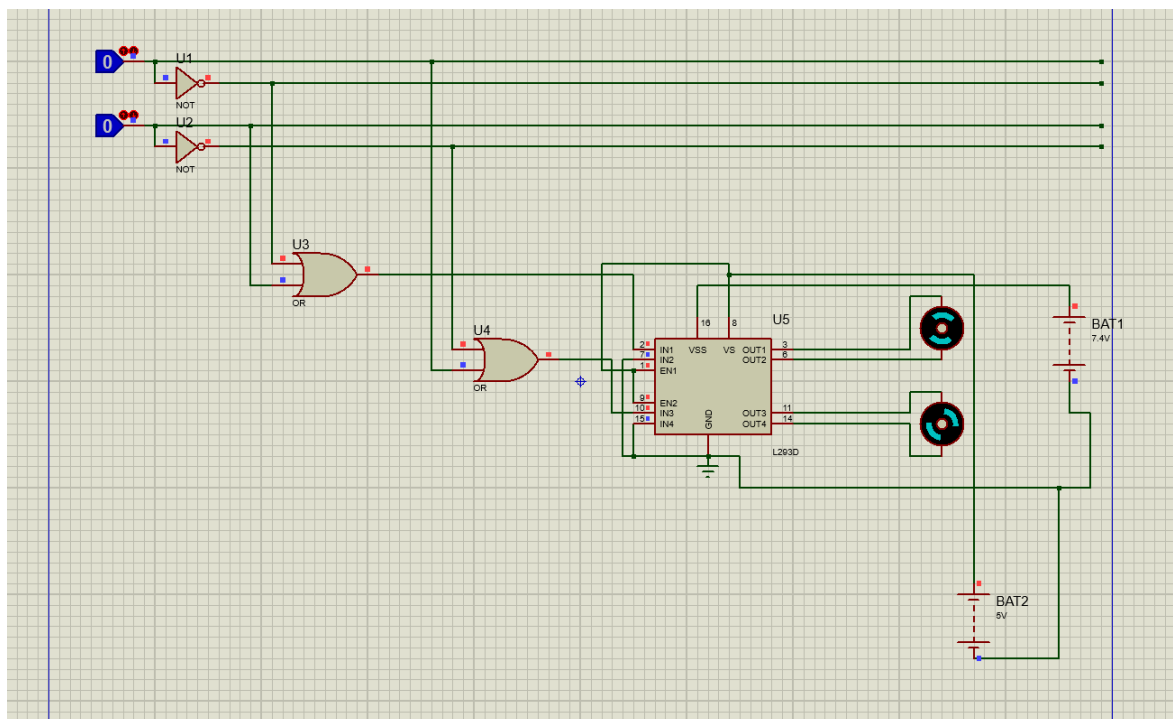
During this week, a functional test prototype of the line-following robot was constructed using **two KY-033 line tracking sensors**, an **L293D motor driver IC**, and **two N20 200 RPM (6 V) DC motors**. The system was powered by **two 3.7 V, 1850 mAh Li-ion batteries connected in series**, providing 7.4 V, and **LM2596 DC buck converter** was used to step down the voltage to 5 V for appropriate circuit operation.

The control logic implemented in this prototype was based on the Boolean expressions derived during the Week 1 discussion session for the two-sensor configuration. After assembling the hardware, the robot was tested on a **black tape line track** to evaluate its performance. The prototype successfully followed the line according to the designed logic. Images of the constructed robot and a video recording of the testing procedure are included below for reference.

Design Evolution: From Basic Logic to Hardware Memory

Version 1.0: Discrete Logic Prototype

- **The Initial Approach:** The first iteration of the robot was built using a straightforward combinational logic circuit utilizing standard discrete gates (separate NOT, AND, and OR ICs).
- **Base Navigation:** The primary logic condition was set so that when both sensors read "white" (indicating the robot was centered perfectly over the line), the output triggered both motors to drive forward. Single-sensor triggers would initiate a left or right turn.
- **The Limitation (The "Lost Line" Problem):** While successful on gentle curves, this version lacked a state machine or memory. If the robot encountered a sharp 90-degree turn and its momentum caused it to overshoot—leaving both sensors on the white surface—the basic logic defaulted back to the "go straight" command. The robot would drive straight off the track, entirely losing the line.



Proteus Simulation:

Simulation Video: [Click here](#)

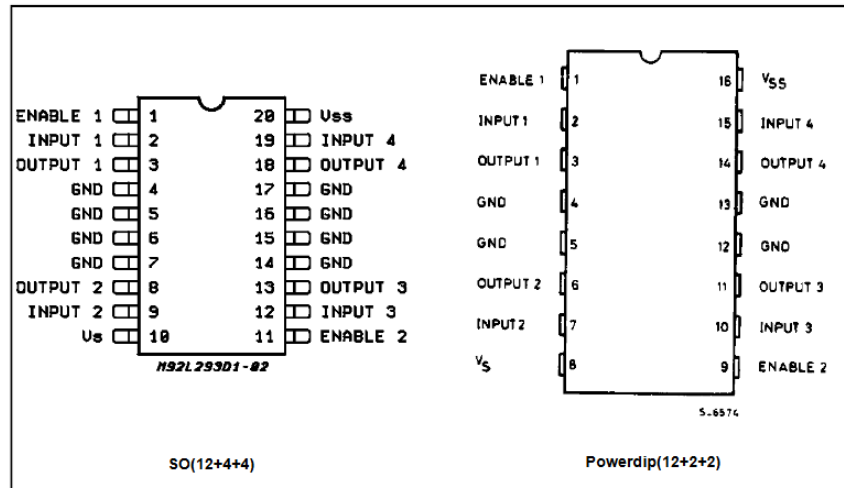


Figure 7: Pin configuration of the L293D IC (Source: [click here](#))

1. Inputs & Logic Gates

- **IR Sensor Modules (x2):** Provide the initial digital HIGH/LOW input signals based on line detection.
- **74HC04 Hex Inverter IC:** Utilized to implement the NOT logic gates, flipping the incoming digital signals from the IR sensors.
- **74HC32 Quad 2-Input OR Gate IC:** Utilized to implement the OR logic gates, which process the conditioned signals to determine the final forward or stop states for the motors.

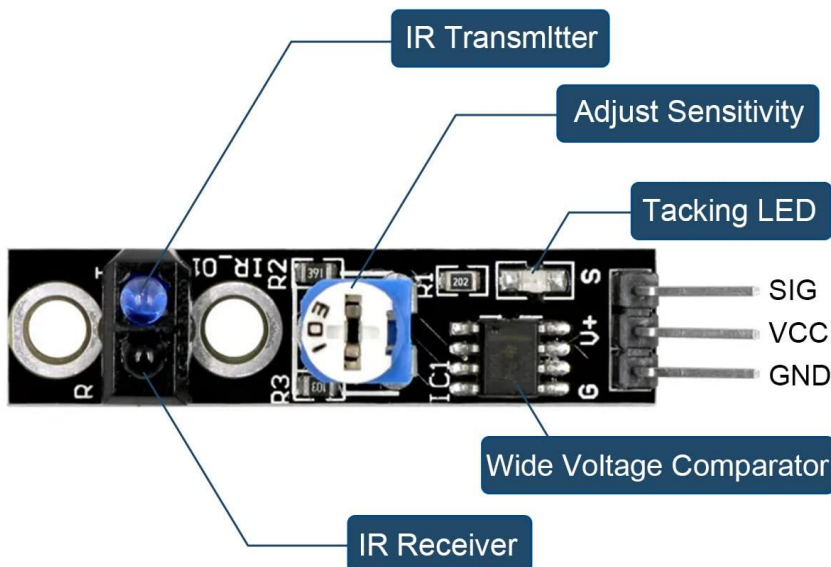
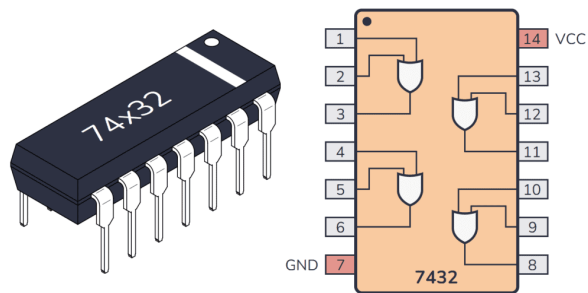


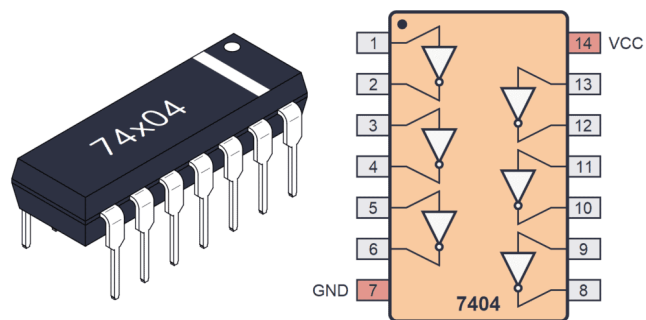
Figure 8:Pin configuration of the KY-033 sensor (Source:[click here](#))

74HC32(OR Gate IC):



Data sheet:[click here](#)

74HC04(NOT Gate IC):



Data sheet:[click here](#)

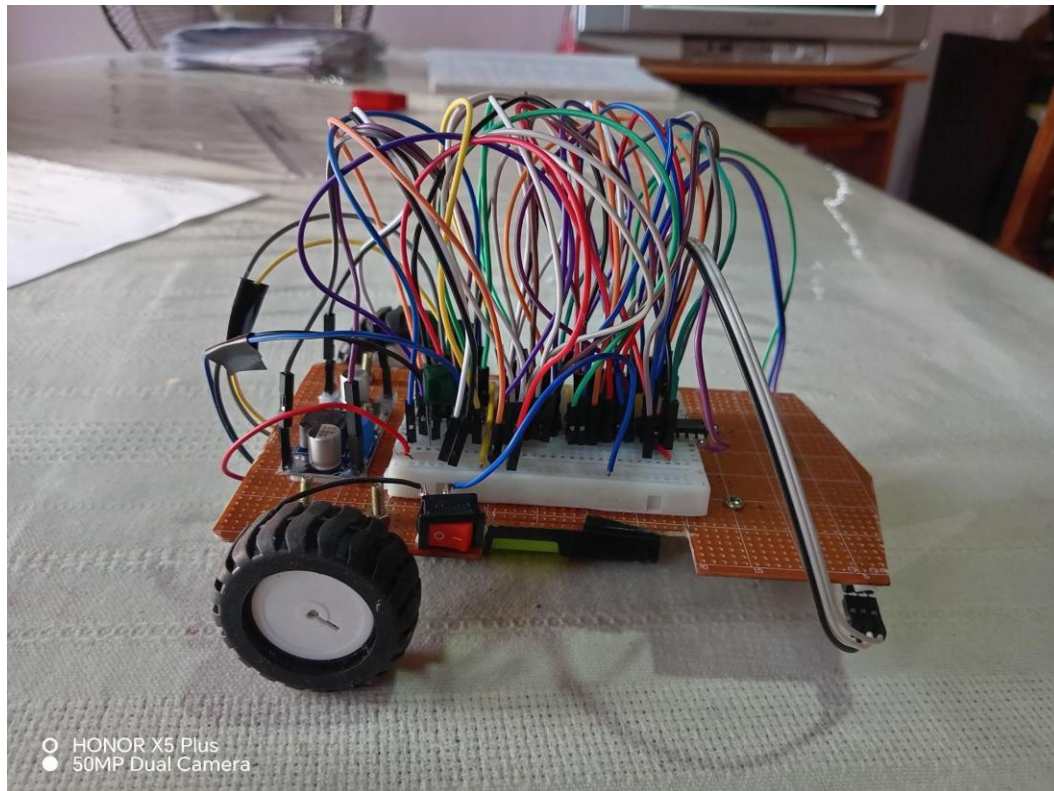
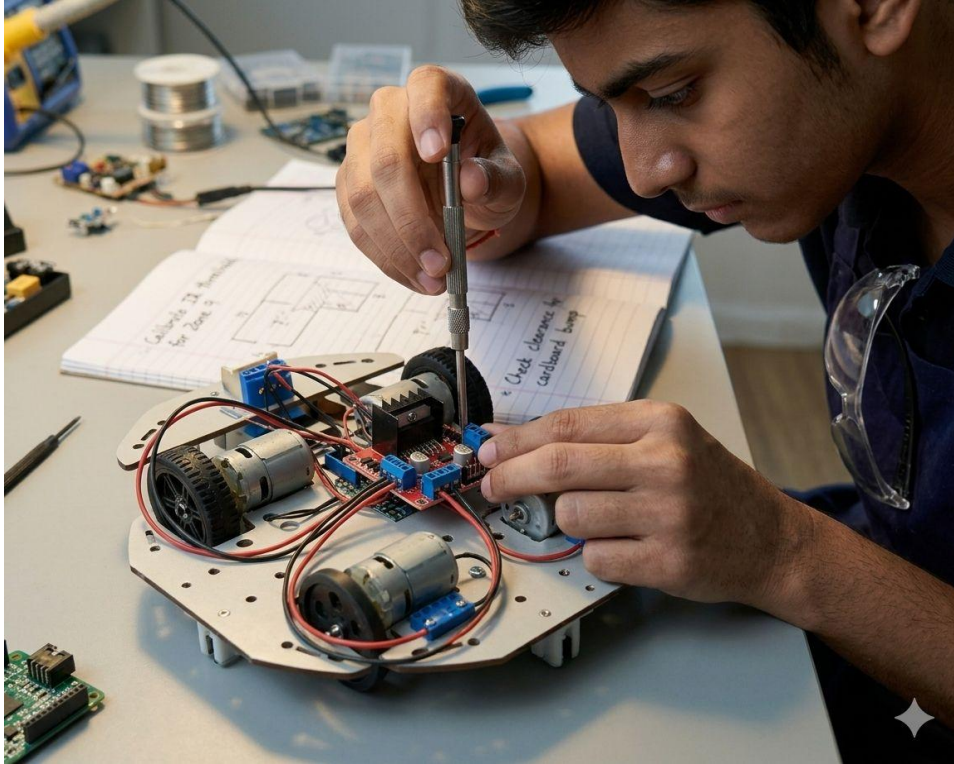


Figure 9:Constructed two-sensor line follower prototype(without memory)

Two-Sensor Line Follower Testing:

Video:[Click here](#)

Day 06



The Flaw (The U-Turn Error): While the initial NOT/OR logic successfully drove the robot forward when both sensors saw white, it failed critically on dashed lines. Because purely combinational logic reacts instantaneously without any context of the past, entering a gap in the line at even a slight angle would cause one sensor to trigger a turn command just a fraction of a second before the other. Without a continuous line to immediately correct this, the robot would get locked into a sharp spin, effectively doing a complete U-turn and driving back down the previous path.

When testing in the final project track we observe that,

Video: [Click here](#)

Solution (Hardware Memory): To solve this, the circuit was upgraded to include two **SR Flip-Flops (Set-Reset Latches)** built entirely from NAND gates.

- **Retaining Context:** The SR latches introduced "state memory" to the hardware. Instead of only knowing what the sensors see *right now*, the robot remembers what the sensors saw *a moment ago*.
- **Bridging the Gap:** When the robot encounters the blank space of a dashed line, the latches hold the last valid navigational command. If the robot was correcting its path as it

entered the gap, it remembers that corrective state, allowing it to smoothly "coast" across the empty space and confidently catch the next segment of the line without erratic spinning.

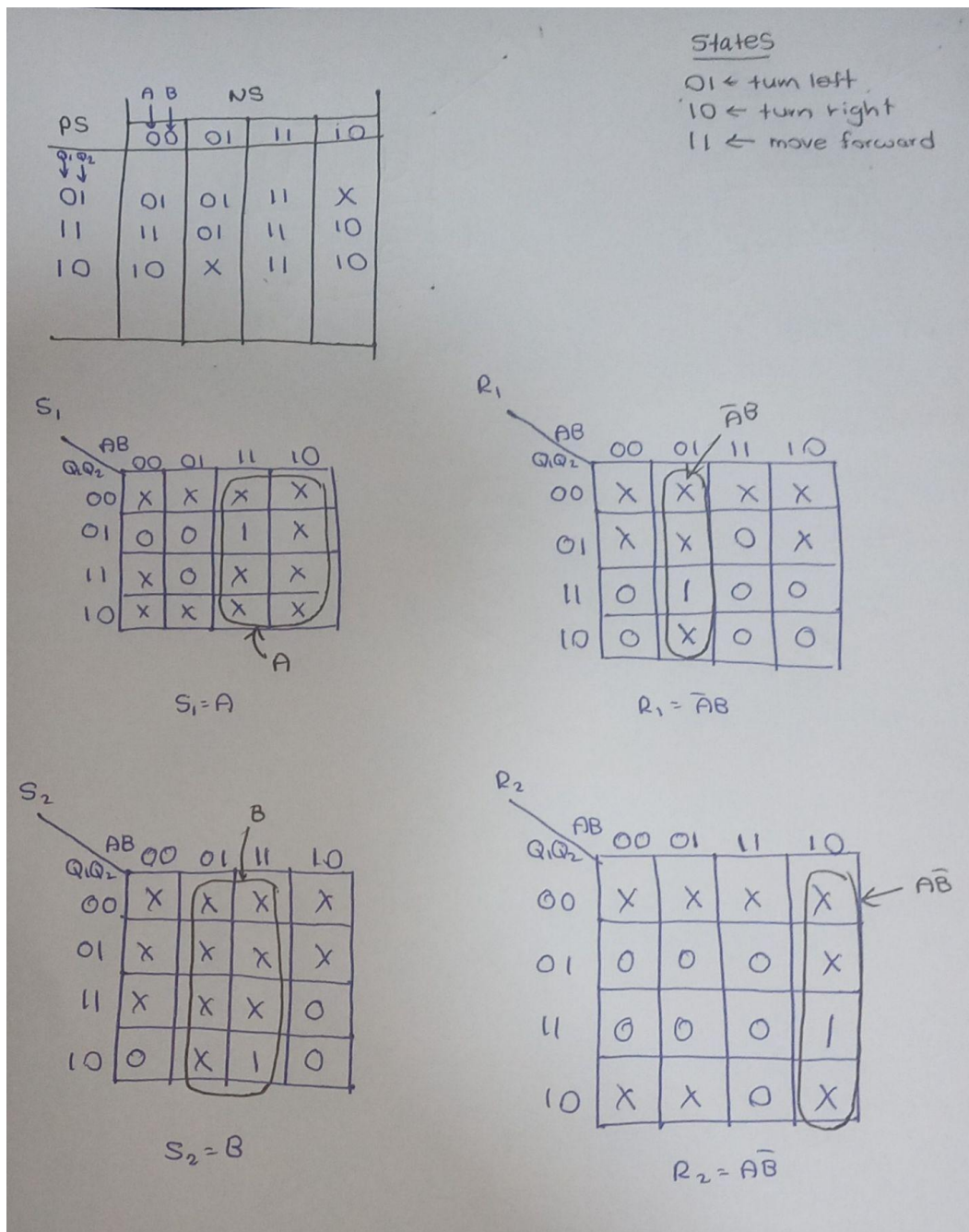
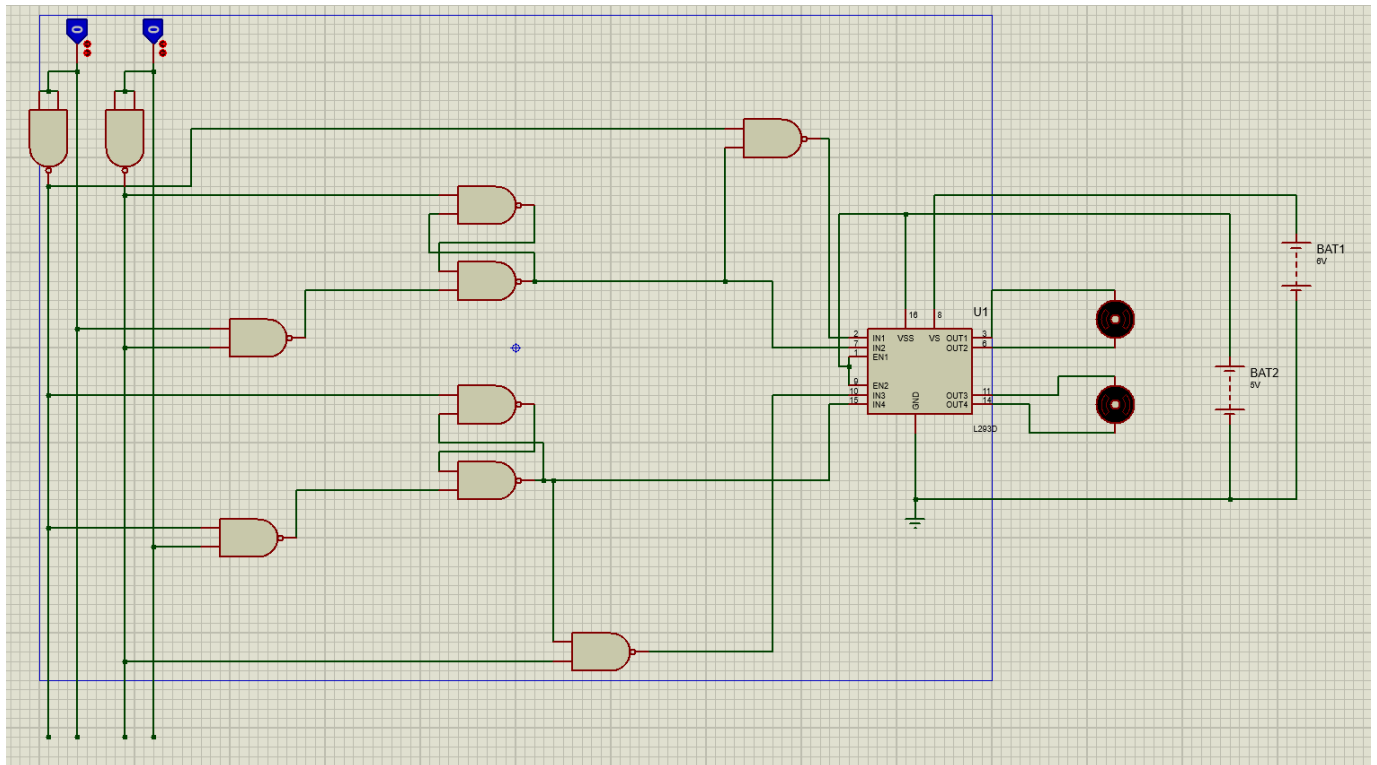


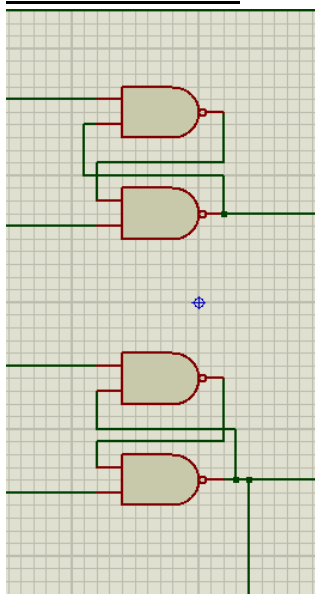
Figure 10: Make a hardware memory using two SR Flip Flops built entirely from NAND gates

Proteus Simulation:

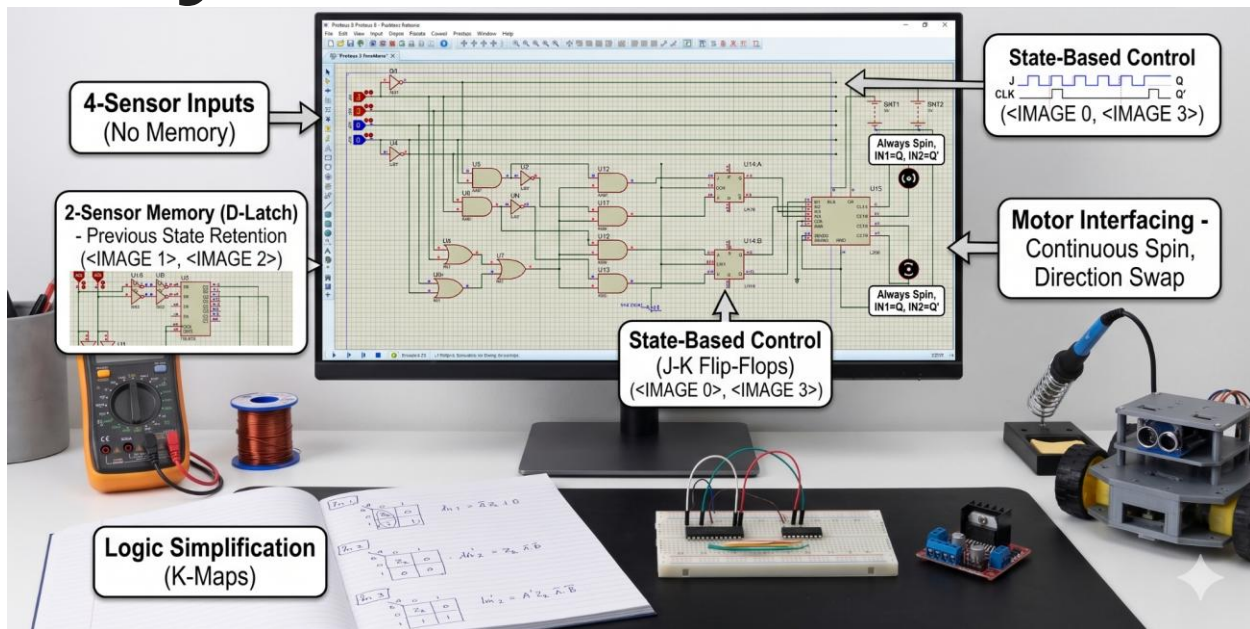


Simulation Video: [Click here](#)

Two S-R Latches:



Day 07



Activity

- Designing and simulating the final, complete logic circuit for a two-sensor line-following robot, featuring an automatic time-delayed stop mechanism, built entirely without the use of a microcontroller.

1. Project Overview & Objective

- **Hardware-Only Autonomous Navigation:** This robot operates entirely on digital logic, completely eliminating the need for a microcontroller or software programming.
- **Problem Solved:** Traditional simple line followers get stuck if both sensors lose the line. Emphasize that your design introduces a "memory" feature to remember the last known position of the line and correct its path.

2. The Core Logic: "Memory" via SR Latches

- **The SR Latch Mechanism:** The two cross-coupled gate pairs in the middle of the schematic are S-R Latches made by using NAND gates.
- **State Retention:** When the robot is exactly on the line (or completely off it, depending on your sensor logic), the latch goes into its "hold" state. It remembers the last sensor that was triggered.
- **Intelligent Correction:** Because of this memory, if the robot overshoots a sharp turn, it knows exactly which direction to spin to find the line again, rather than just stopping or spinning blindly.

3. Universal Logic Implementation (NAND Gates)

- **Universal Gate Concept:** the *entire* control logic was designed using only NAND gates.
- **Cost & Space Efficiency:** By using a universal gate minimizes the number of different integrated circuit (IC) chips required (you likely used standard logic ICs like the 74HC00 or CD4011). This reduces BOM (Bill of Materials) cost and saves space on the PCB.
- **Inverter Design:** Note that the very first stage (the two NAND gates connected to the logic inputs) are simply NAND gates wired to act as NOT gates (inverters) to condition the sensor signals.

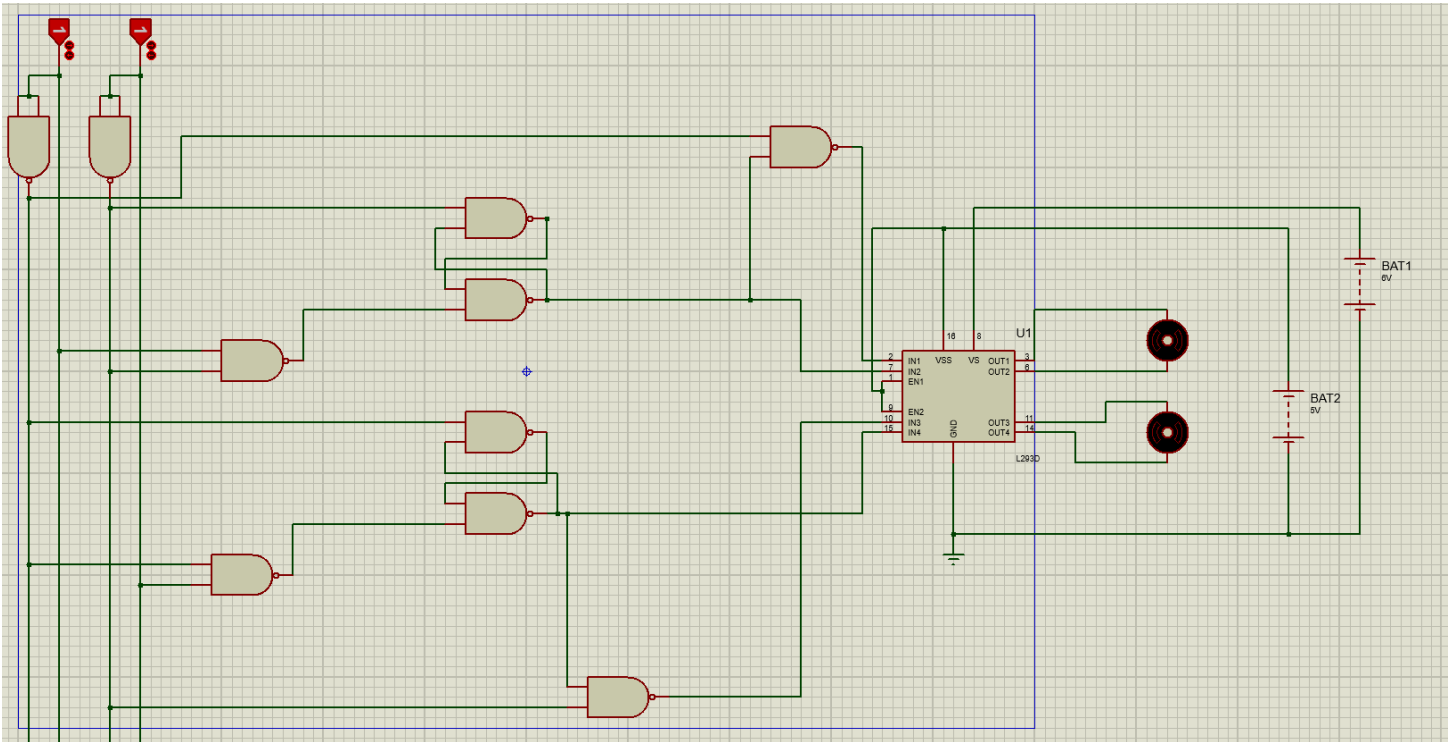
4. Motor Drive & Actuation (L293D)

- **H-Bridge Motor Driver:** Detail the use of the L293D IC. It allows the low-current digital logic to control the higher-current DC motors safely.
- **Differential Steering:** Steering is achieved by driving the two motors independently (e.g., stopping or reversing one motor while driving the other forward to execute a turn).
- **Enable Pin Utilization:** The logic outputs don't just control the IN1/IN2 direction pins, but are also routed to the Enable pins (EN1, EN2) to cut power to the motors entirely when logic dictates.

5. Advanced Power Management Strategy

- **High-Density Power Source:** Power is supplied by two 18650 Lithium-ion batteries in series (providing a nominal ~7.4V). Highlight that 18650s are chosen for their high discharge capacity, ensuring the motors have plenty of current available on demand, especially during stall conditions or sharp turns.
- **High-Efficiency Voltage Regulation:** Instead of using an inefficient linear regulator (like an LM7805 that wastes excess voltage as heat), a **buck converter** is used to step down the 7.4V to a stable 5V for the digital logic. This demonstrates an understanding of power efficiency and thermal management.
- **Decoupled Logic and Motor Rails:** Explain the specific wiring to the motor driver:
 - **Motor Power (V_s):** Receives the unregulated, full ~7.4V battery voltage directly. This maximizes the torque and speed of the motors.
 - **Logic Power (V_ss):** Receives the cleanly regulated 5V from the buck converter.
 - **Reason behind this:** By separating these power rails, the sensitive NAND gate logic is protected from "brownouts" or voltage spikes caused by the heavy, inductive load of the motors turning on and off.

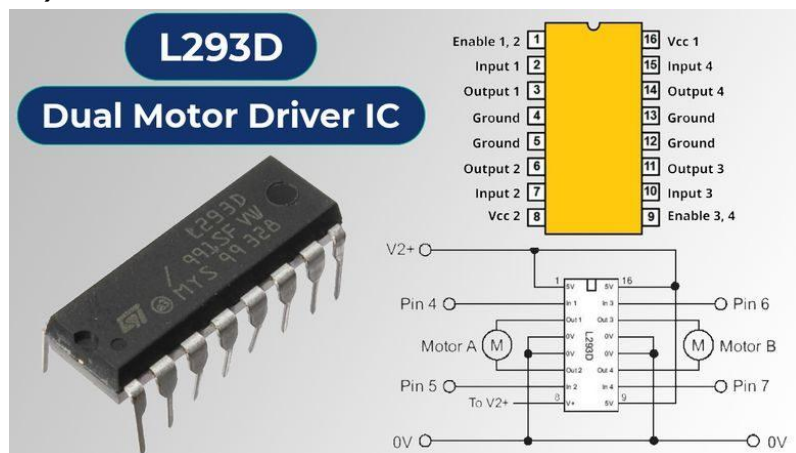
Final Logic Circuit Simulation:



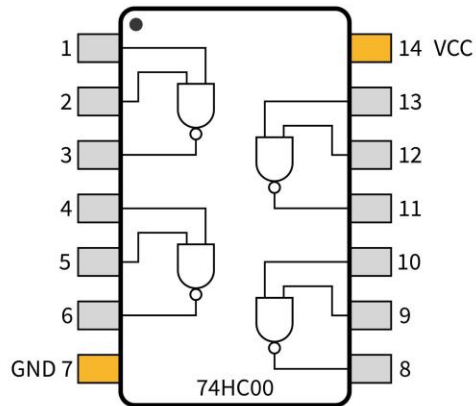
Simulation Video Link: [Click here](#)

Components:

1) L 293D Motor Drive IC

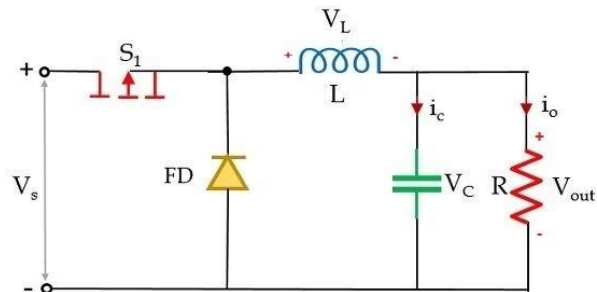


2)74HC00 IC(NAND Gate IC)



JOTRIN.COM

3)Buck Converter



Buck Converter Circuit diagram

How2Electronics

Pin configuration of the LM2596 DC buck converter (Source:[click here](#))

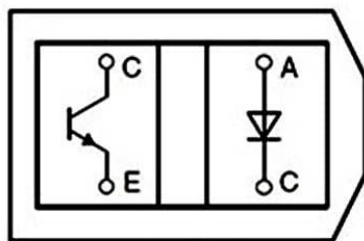
4)300 RPM Motors(6V)



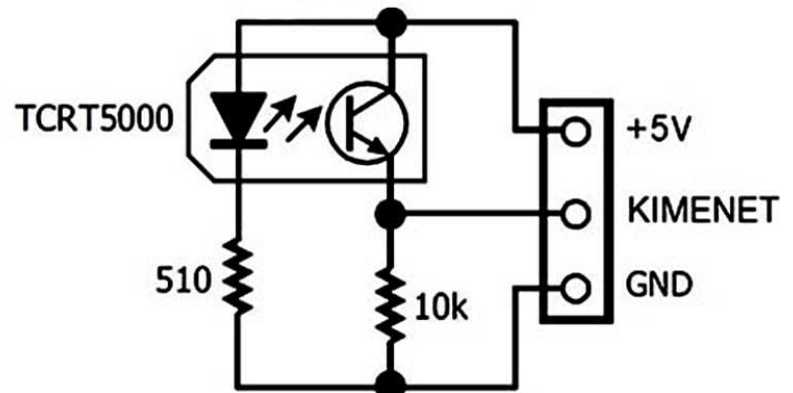
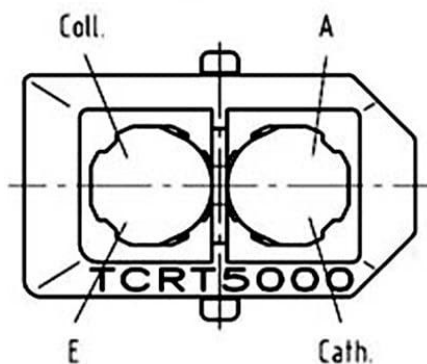
5)Li-Ion Battery(3.7V)



TCRT5000 Reflective Optical Sensor



Top view



Day 08



Activity

- According to the final logic circuit(with 2 S-R Latches),we started to implement the breadboard prototype using jumper wires.

Step-by-Step Breadboard Implementation

1. Power Rail Preparation & Isolation

- **Establish the Logic Rail (5V):** Connect the 5V output and Ground from the buck converter to the breadboard's power rails. In this design, to prevent motor noise from resetting the logic, one side of the breadboard's power rails is dedicated strictly to the clean 5V logic supply (V_SS).

- **Establish the Motor Rail (7.4V):** Route the raw battery positive terminal to an isolated section or separate rail specifically for the motor driver's power input (V_S), ensuring it does not mix with the 5V logic rail. Tie all Grounds together.

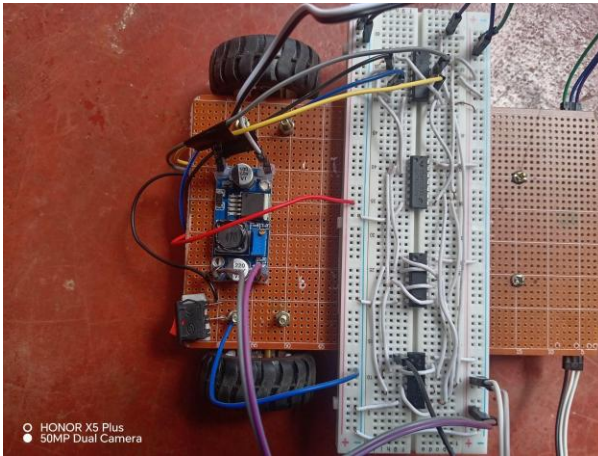


Figure 11: Connect buck converter

2. IC Placement

- Carefully insert the Integrated Circuits (ICs) across the center trench of the breadboard to prevent shorting opposite pins.
- **Components:** Place the three Quad 2-Input NAND gate ICs (74HC00) needed to accommodate the 10 total NAND gates required by the schematic. Place the **L293D Motor Driver IC** toward the rear of the breadboard, closer to the physical motors.

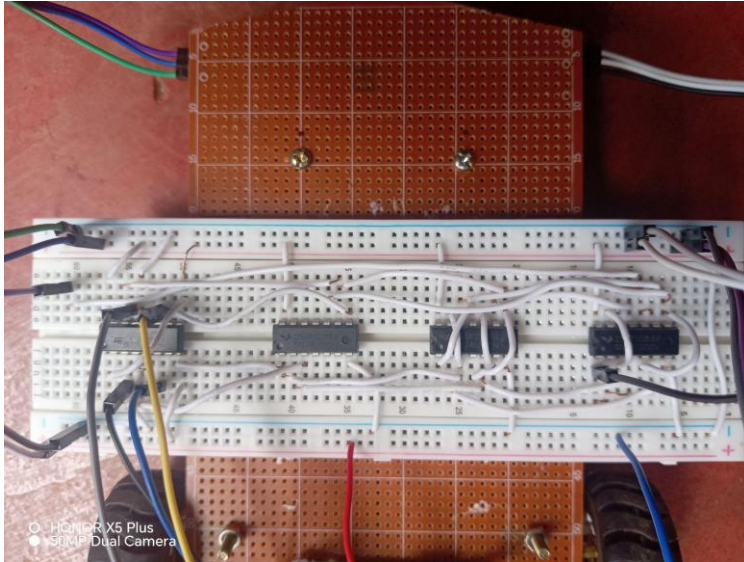


Figure 12: IC placement on breadboard

3. IC Power and Grounding

- Wire the power connections for all logic chips: Connect the V_{CC} pin (Pin 14) of each NAND IC to the clean 5V rail, and the GND pin (Pin 7) to the common ground rail.
- **L293D Power Routing:** Carefully wire the L293D according to the split-power strategy:
 - Connect Pin 16 (V_{SS}) to the **5V logic rail**.
 - Connect Pin 8 (V_S) to the **7.4V raw battery rail**.
 - Connect the four center ground pins (Pins 4, 5, 12, 13) to the common ground rail.

4. Sensor Input Conditioning (Inverters)

- Connect the signal wires from the left and right IR sensor modules to the inputs of the first two NAND gates.
- Tie both inputs of each of these specific NAND gates together. This configures them to act as NOT gates (inverters), successfully conditioning the raw sensor data for the rest of the circuit.

5. Assembling the Hardware Memory (SR Latches)

- Use short, flat jumper wires to construct the cross-coupled SR latches.
- Route the output of the upper NAND gate into one input of the lower NAND gate, and route the output of the lower NAND gate back into the input of the upper one.
- *Best Practice implemented:* Keeping these feedback loops as short as possible minimizes parasitic capacitance and prevents logic oscillation.

6. Motor Control Routing

- Route the final calculated logic outputs from the NAND network to the directional control pins of the L293D:
 - Connect the left logic outputs to **IN1** and **IN2**.
 - Connect the right logic outputs to **IN3** and **IN4**.
- **Enable Pins:** Route the necessary logic signals to the **EN1** and **EN2** pins to ensure the motors only receive power when a valid drive state is active, saving battery life and preventing motor stall jitter.

7. Final Actuation Connections

- Connect the positive and negative terminals of the Left DC Motor to **OUT1** and **OUT2** on the L293D.
- Connect the Right DC Motor to **OUT3** and **OUT4**.
- *Testing Phase:* Perform a manual continuity check on the power rails before connecting the 18650 batteries to prevent accidental logic burnout

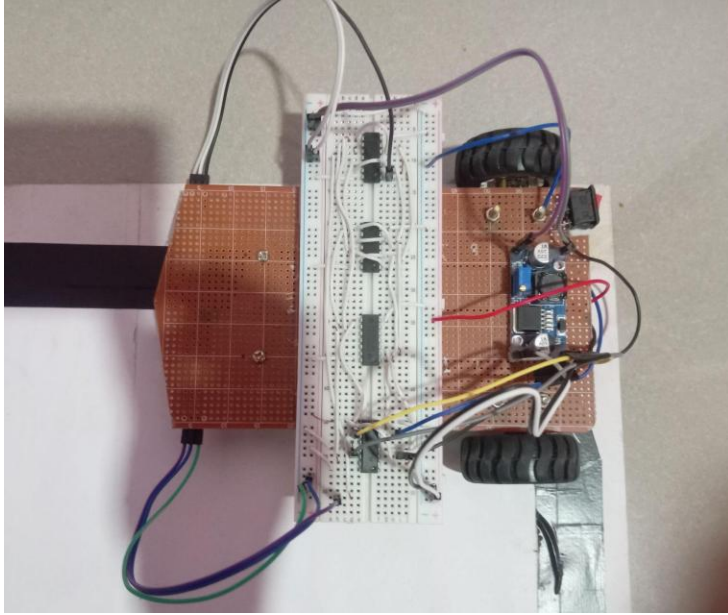


Figure 13: Top view

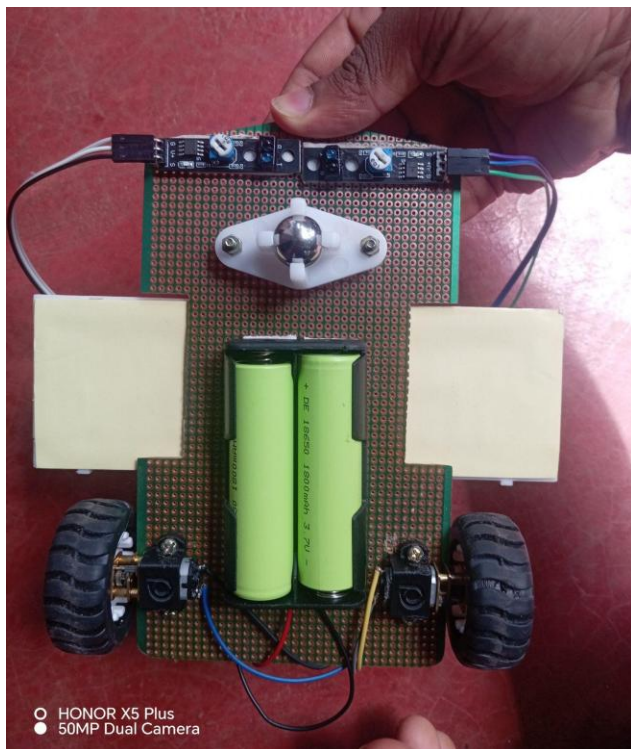
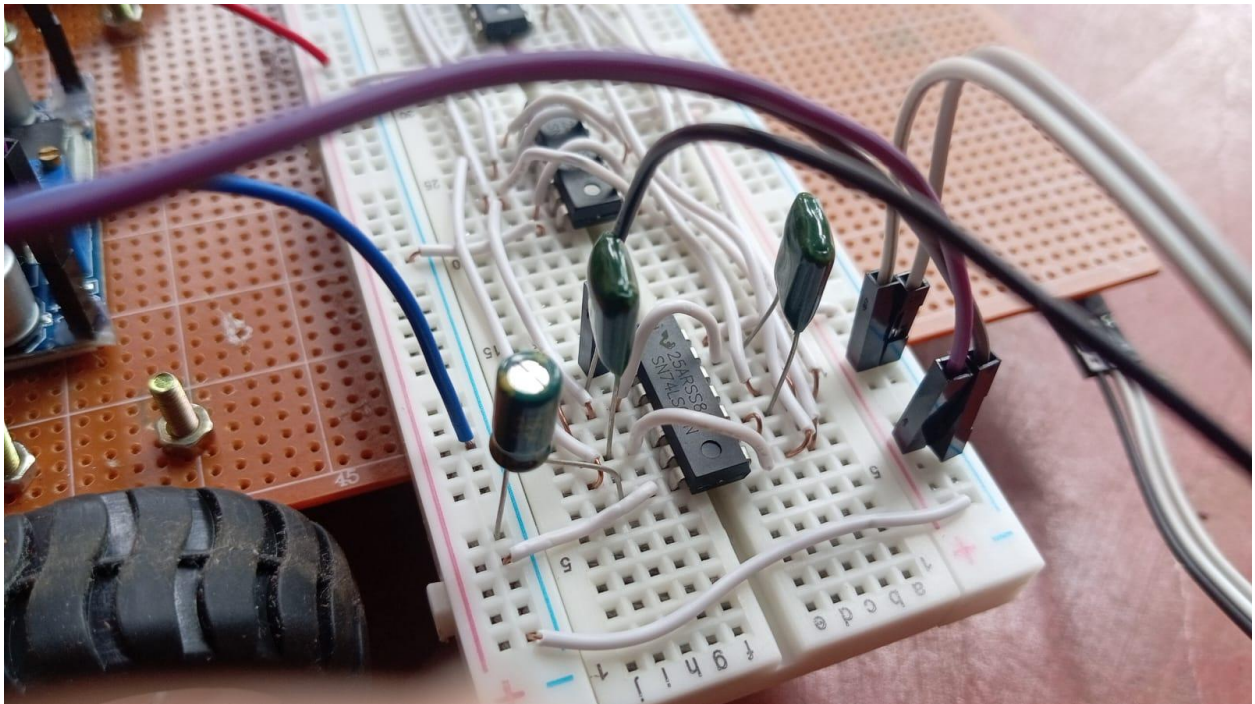


Figure 14: Bottom view

Day 09

Activity

- To transition the robot from a theoretical prototype to a stable physical model, decoupling and snubber capacitors were added to the circuit to manage power surges and electrical noise.



Logic Power Stabilization (VCC to GND):

- **Component:** Added a **10 μ F - 100 μ F electrolytic capacitor** across the logic power rails.
- **Purpose:** Acts as a local energy reservoir to prevent voltage drops (brownouts) when motors pull sudden current. Ensures the 5V logic remains stable so the SR latches don't accidentally lose their "memory."

Motor Noise Suppression:

- **Component:** Added two **0.1 μ F (104) film/ceramic capacitors** near the logic/motor inputs.
- **Purpose:** Act as low-pass filters to absorb high-frequency electromagnetic interference (EMI) and sudden voltage spikes generated by the brushed DC motors.

Mechanical Smoothness: * ResuEstimated Capacitor Values:

- **1x Electrolytic Capacitor (Cylindrical):** Likely **10 μ F to 100 μ F**. Used for bulk power stabilization.
- **2x Film/Ceramic Capacitors (Green):** Likely **0.1 μ F** (often marked "104"). This is the industry standard for filtering high-frequency noise.

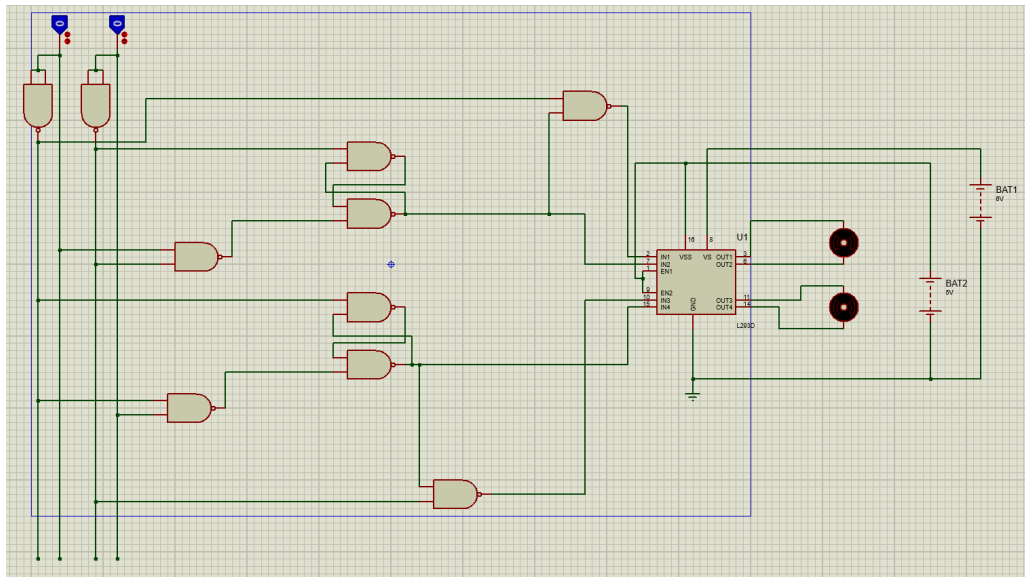
It: By absorbing sharp electrical transients, these capacitors soften the instantaneous "jerk" when the motor driver switches directions. This provides smoother physical cornering and less wheel-slip without reducing the robot's top speed.

Day 10

Activity

- Analysing the problems we faced throughout the process and the solutions we got, driving the line following robot on the final project track.

Final Logic Circuit:



Line Following Robot:

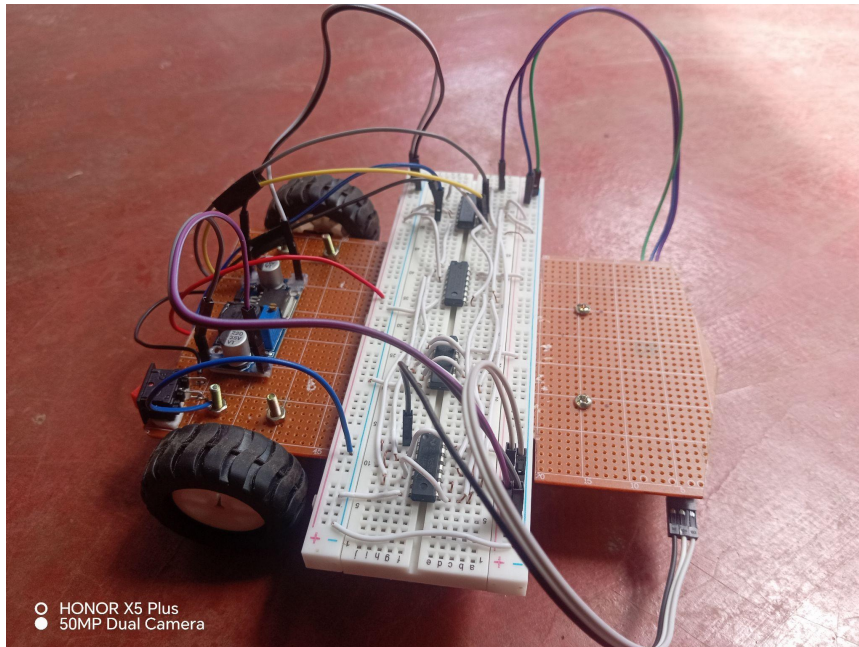


Figure 15: Driving on the Home Track

Video: [Click here](#)

Problems we faced throughout the process and Solutions that we taken

During the making, testing, and driving phases of this purely hardware-driven robot, several physical and logical challenges were encountered and methodically solved:

- **The "Dashed Line / U-Turn" Error (Logic Limitation):**
 - **Problem:** The initial prototype (without memory) utilized basic combinational logic (NOT/OR gates). When encountering a dashed line or a sharp 90-degree turn where both sensors briefly lost the line, the robot lacked context of its previous state. It would either drive straight off

the track or enter a rapid spin, effectively performing a U-turn and going back the way it came.

Video: [Click here](#)

- **Solution:** Redesigned the entire control circuit using universal NAND gates to implement **SR Flip-Flops (Latches)**. This introduced "hardware memory", allowing the robot to remember the last known position of the line and smoothly correct its path across gaps or sharp corners.

- **Logic Brownouts & Motor Noise (Signal Integrity):**
 - **Problem:** During test drives, the brushed DC motors generated severe electromagnetic interference (EMI) and massive current spikes when instantly reversing direction. This caused micro-voltage drops (brownouts) on the logic rail, randomly resetting the SR latches and causing erratic twitching.
 - **Solution:** Implemented hardware smoothing. Added a **10 μ F–100 μ F electrolytic capacitor** across the logic power rails to act as a local energy reservoir, preventing brownouts. Added **0.1 μ F (104) ceramic capacitors** near the inputs to filter out high-frequency motor noise, successfully stabilizing the "brain" of the robot.

- **Power Delivery & Thermal Inefficiency (Power Management):**
 - **Problem:** Dropping the ~7.4V battery voltage down to 5V for the logic ICs using standard linear voltage regulators caused excess heat dissipation and drained the batteries too quickly during heavy testing.

- **Solution:** Integrated a high-efficiency **buck converter** to step down the voltage to a stable 5V for the logic (VSS) without thermal waste. Simultaneously, the raw 7.4V from the 18650 Li-ion batteries was routed directly to the L293D motor supply (VS) to ensure maximum torque without starving the logic chips.
- **Overshooting & Ambient Light (Physical Tuning):**
 - **Problem:** Because the motors run fast on raw 7.4V, the robot occasionally overshoot sharp corners before the logic could react. Additionally, ambient room lighting caused false sensor readings.
 - **Solution:** Physically extended the IR sensor array further ahead of the chassis to give the circuit crucial extra milliseconds to react to a turn. To fix light interference, the IR module potentiometers were meticulously tuned to the specific room environment.

Project Conclusion

This project successfully demonstrates the design and assembly of a highly capable autonomous line-following robot using purely fundamental digital electronics. By eliminating the reliance on a microcontroller and software programming, this build highlights a deep, practical understanding of universal logic gates, state machines, and real-world circuit integration.

Moving from a basic combinational logic prototype to an advanced, state-retaining design using SR Latches proved the vital importance of iterative engineering. Furthermore, integrating a buck converter, decoupling capacitors, and split-rail power management transformed a theoretical breadboard schematic into a robust, high-performance

physical machine capable of handling the harsh electrical realities of motorized robotics.

Final Demonstration Video(On Project Track) : [Click here](#)